密码学引论第一次实验

江李阳，王建一，邱泽辉

March 25, 2026

1 实验一

1.1 实验目的

1. 理解分组密码中 S 盒的非线性混淆原理。
2. 掌握差分分布表的构造方法。

1.2 实验原理

1.2.1 差分分布表定义

对于 $6 \to 4$ 位的 S 盒，输入差分 α 与输出差分 β 的关联强度由 $N_S(\alpha, \beta)$ 衡量。其数学定义为：在一个输入空间内，满足特定输入差分导致特定输出差分的输入值数量。

$$N_S(\alpha, \beta) = \#\{X \in \{0,1\}^6 \mid S(X) \oplus S(X \oplus \alpha) = \beta\}$$

其中 X 遍历所有 64 种输入可能。

1.3 实验数据分析

1.3.1 差分分布表 (DDT) 概览

本实验生成的 S1 盒 DDT 规模为 64×16。表中的数值反映了特定差分传播的频数。具体差分分布表见附录 A。

1.3.2 高概率差分路径

实验数据如下表所示：

Table 1: S1 盒高概率差分路径汇总表

排名	计数	概率	(输入差分, 输出差分) 列表
1	64	1.000000	(0x00, 0x0)
2	16	0.250000	(0x34, 0x2)
3	14	0.218750	(0x03, 0x0), (0x0C, 0xE), (0x10, 0x7), (0x1D, 0xE), (0x1E, 0x4), (0x24, 0x8), (0x24, 0x9), (0x29, 0x9), (0x2A, 0x9), (0x35, 0x8), (0x35, 0xE), (0x3E, 0x7)
4	12	0.187500	(0x01, 0xA), (0x02, 0xC), (0x05, 0xC), (0x06, 0xF), (0x08, 0x3), (0x09, 0xF), (0x0B, 0xF), (0x10, 0xB), (0x1C, 0x7), (0x20, 0x5), (0x20, 0xF), (0x24, 0x0), (0x25, 0x3), (0x28, 0x0), (0x28, 0x6), (0x2B, 0x0), (0x2E, 0xC), (0x30, 0x4), (0x34, 0x7), (0x37, 0x2), (0x39, 0x4), (0x3C, 0x4), (0x3C, 0xB), (0x3D, 0xD)

1.3.3 概率传播种类统计

实验统计显示，该 S 盒共有 9 种不同的非零计数值，对应 9 种传播概率：

- **低概率路径**：计数为 2, 4, 6，对应的最低传播概率为 0.0312。
- **中概率路径**：计数为 8, 10, 12。
- **高概率路径**：计数为 14, 16，最高传播概率达到 0.2500（即 16/64）。

1.3.4 差分碰撞分析（输入非 0 输出为 0）

实验结果表明，当输入差分 $\alpha \neq 0$ 时，输出差分 β 可能为 0。**原因分析：** DES 的 S 盒是一个从 6 位到 4 位的压缩映射。根据抽屉原理（Pigeonhole Principle），必然存在多个不同的输入映射到相同的输出。即：

$$\exists X_1 \neq X_2, \text{ s.t. } S(X_1) = S(X_2) \implies S(X_1) \oplus S(X_2) = 0$$

当 $\alpha = X_1 \oplus X_2$ 时，该输入差分会导致输出差分为 0。这在差分分析中意味着该 S 盒在该路径下是“不活动的”。

2 实验二

2.1 实验目的

1. 验证 AES 算法在 3 轮加密下的平衡性（Balance）现象。
2. 探究轮数增加对积分特性消失的影响。
3. 分析列混淆变换（MixColumns）在密码扩散中的核心作用。

Table 3: S1 盒差分碰撞分布表 ($\alpha \neq 0, \beta = 0$)

输入差分	碰撞频数	输入差分	碰撞频数	输入差分	碰撞频数
0x03	14	0x25	6	0x2C	4
0x24	12	0x2D	6	0x31	4
0x28	12	0x2E	6	0x32	4
0x2B	12	0x39	6	0x33	4
0x09	10	0x3A	6	0x3E	4
0x22	10	0x05	4	0x3F	4
0x27	10	0x17	4	0x07	2
0x0D	6	0x1B	4	0x0B	2
0x11	6	0x1D	4	0x0F	2
...	...	0x29	4	...	...

2.2 实验原理

2.2.1 积分特性与 Λ-集合

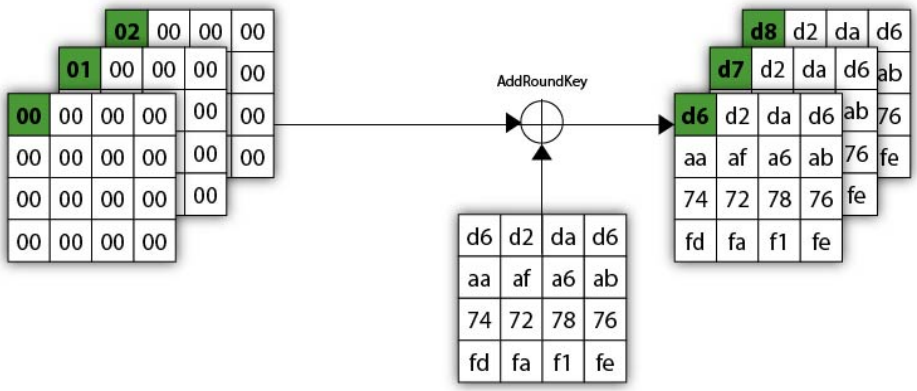
在对 AES 的积分攻击中，我们构造一个特殊的明文集合，称为 Λ-集合。在该集合中，选定的字节遍历 $0 \ldots 255$（称为 Active 字节），而其余字节保持固定（称为 Constant 字节）。[1, 2] [1]

对于一个包含 256 个明文的 Λ-集合，在经过 3 轮 AES 加密后，其密文状态满足：

$$\bigoplus_{i=1}^{256} C^{(i)} = 0 \tag{1}$$

下面展开具体说明

2.2.2 AddRoundKey 不影响活跃字节

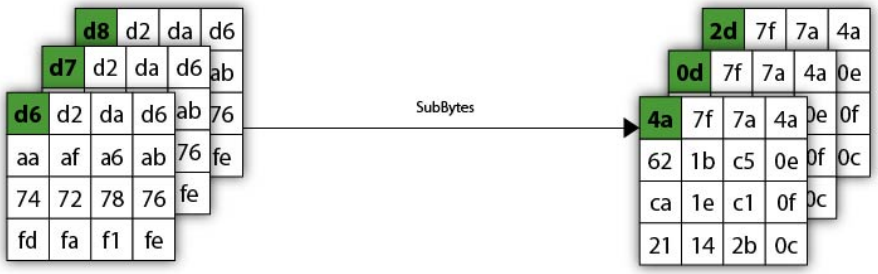


首先一开始先将明文和密钥进行 AddRoundKey 操作，之后进入第 1 轮。异或是线性运算，当活跃字节遍历 $0 \ldots 255$ 时，输出字节也会完整遍历 $0 \ldots 255$。所以 AddRoundKey

[1]参考文献 2 的链接目前已失效（或许是作者没有维护），但是可以通过互联网档案馆（Wayback Machine）访问：https://web.archive.org/web/20240101000000*/https://www.davidwong.fr/blockbreakers/index.html。

操作不影响活跃性。[2]

2.2.3 SubBytes 不影响活跃字节

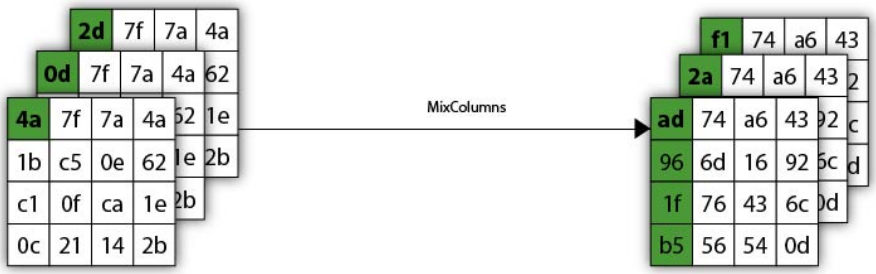


SubBytes 是一个 $\{0,1\}^8 \to \{0,1\}^8$ 的映射。因此，活跃字节在经过 SubBytes 后仍然保持活跃，继续遍历 $0 \ldots 255$。

2.2.4 ShiftRows 不影响活跃字节数量，但是可能影响活跃字节的位置

ShiftRows 只是对状态矩阵的行进行循环移位，不改变字节的值。因此，活跃字节在经过 ShiftRows 后仍然保持活跃。由于第一行不进行移位，所以文献中并未改变活跃字节位置。但是对于实验中的位置，活跃字节在 $(1,0)$ 的情况，经过 ShiftRows 后它会移动到 $(1,3)$，但仍然是活跃的。只是换了一个位置。

2.2.5 列混淆（MixColumns）的作用



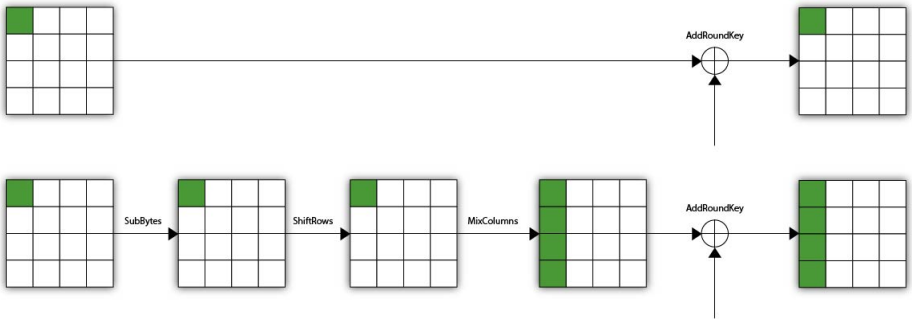
MixColumns 是 AES 中唯一的线性扩散层。它负责在列内进行线性组合, 他相当于一个矩阵乘法

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \cdot x_0 + 3 \cdot x_1 + x_2 + x_3 \\ x_0 + 2 \cdot x_1 + 3 \cdot x_2 + x_3 \\ x_0 + x_1 + 2 \cdot x_2 + 3 \cdot x_3 \\ 3 \cdot x_0 + x_1 + x_2 + 2 \cdot x_3 \end{bmatrix}$$

[2]实验中要求的活跃字节是在矩阵坐标 $(1,0)$ 处的字节，即第二行第一列的字节。但是由于参考文献 2 的图片绘制精美，此处我想直接使用该文献中的图片来说明 AddRoundKey 不影响活跃字节的原因。

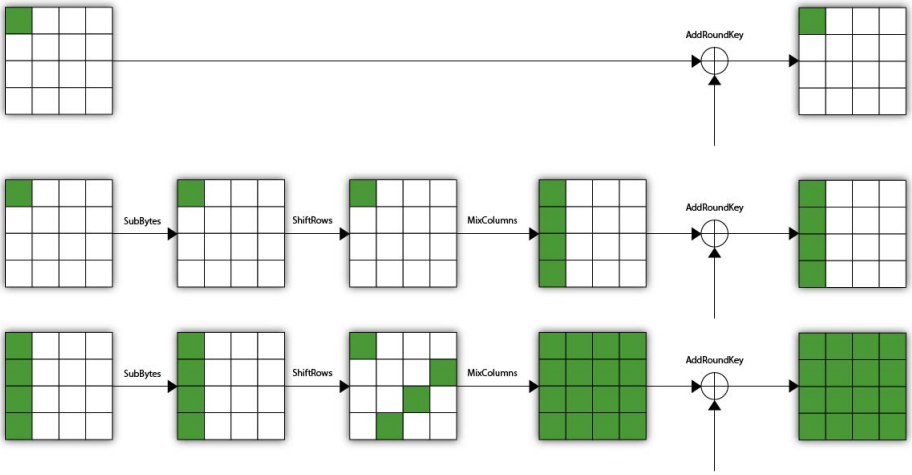
我们的输入是第一列的活跃字节 x_0 和其他列的常数字节。经过 MixColumns 后，第一列的输出将是所有输入字节的线性组合，只要包含 x_0，都会变成活跃字节，因为当 x_0 遍历 $0 \ldots 255$ 时，输出也会完整遍历 $0 \ldots 255$。因此，MixColumns 不会破坏活跃字节的性质，但它会将活跃字节的影响扩散到整列。

2.2.6 第一轮结束结果



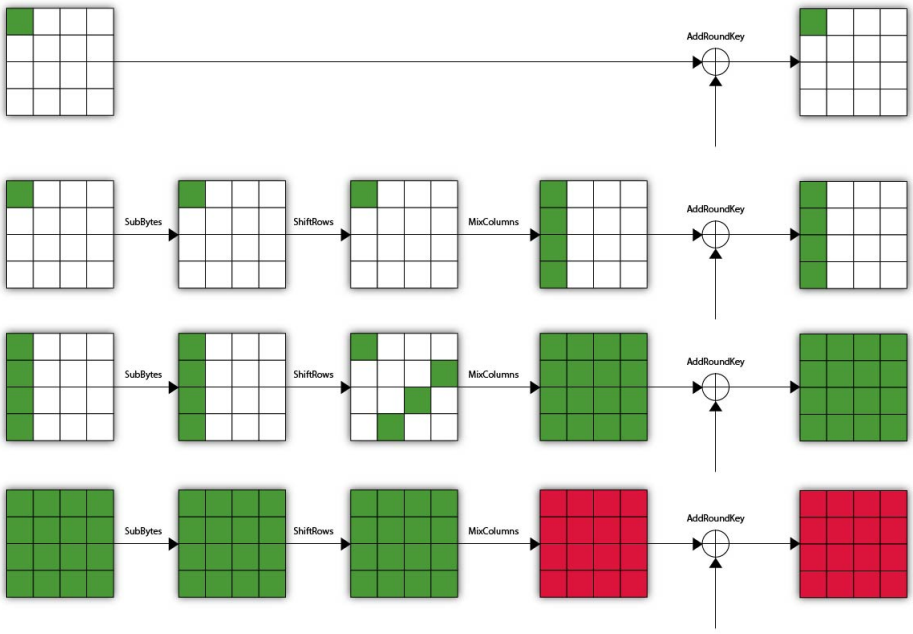
经过第一轮加密，我们得到了第一列都为活跃字节的状态。

2.2.7 第二轮结束结果



接下来进入第二轮，第二轮的 SubBytes 和 ShiftRows 不会改变活跃字节的数量，但 ShiftRows 会将第一列的活跃字节扩散到其他列，然后 MixColumns 会将这些活跃字节的影响扩散到整列。最终，在第二轮结束时，状态矩阵中的所有字节都变成了活跃字节。

2.2.8 第三轮结束结果



到了第三轮发生了变化，在 MixColumns 之后，活跃字节的线性组合，就不一定还是活跃字节了，下面给出证明

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} x_{i,0} \\ x_{i,1} \\ x_{i,2} \\ x_{i,3} \end{bmatrix} = \begin{bmatrix} 2 \cdot x_{i,0} + 3 \cdot x_{i,1} + x_{i,2} + x_{i,3} \\ x_{i,0} + 2 \cdot x_{i,1} + 3 \cdot x_{i,2} + x_{i,3} \\ x_{i,0} + x_{i,1} + 2 \cdot x_{i,2} + 3 \cdot x_{i,3} \\ 3 \cdot x_{i,0} + x_{i,1} + x_{i,2} + 2 \cdot x_{i,3} \end{bmatrix}$$

根据活跃字节的定义，满足

$$\bigoplus_{i=1}^{256} x_{i,j} = 0$$

2.3 现象分析与讨论

2.3.1 3 轮与 4 轮现象分析

当轮数为 3 时，由于 S 盒的非线性复杂度尚未完全积累，输出函数 $f(x)$ 的代数次数 $deg(f) < 8$。根据引理，在 $GF(2^8)$ 上：

$$\sum_{x \in GF(2^8)} x^d = 0, \quad \text{for } 0 \leq d < 255 \tag{2}$$

因此异或和为 0。当进入第 4 轮，代数次数迅速饱和，平衡性随之打破。

2.3.2 删除 MixColumns 的影响

实验发现，若删除 MixColumns，即使加密至 31 轮，平衡性依然保持。这是因为没有 MixColumns 时，每个字节的变换仅由 S 盒复合组成。由于 S 盒是置换（Permutation），多个置换的复合依然是置换。如果输入遍历了 $0 \ldots 255$，输出必然也完整遍历 $0 \ldots 255$。已知：

$$\bigoplus_{i=0}^{255} i = 0 \tag{3}$$

故该现象在无扩散层时会永久保持。

2.4 实验结论

本实验成功验证了 AES 的积分特性。实验证明，列混淆（MixColumns）不仅提供了列内的线性扩散，更是打破代数平衡性、防止积分攻击的关键组件。没有 MixColumns 的 AES 即使轮数再多，也无法防御结构化攻击。

2.5 实验过程与结果

2.5.1 实验设置

构造 3 组 Λ-集合，每组包含 256 个明文，在矩阵坐标 $(1,0)$ 处遍历。分别测试 3 轮、4 轮以及无 MixColumns 情况下的异或和。我们选取的明文如下：

$$P_0 = \begin{bmatrix} 123 & 34 & 115 & 178 \\ * & 195 & 84 & 74 \\ 198 & 133 & 232 & 32 \\ 225 & 167 & 181 & 207 \end{bmatrix}$$

$$P_1 = \begin{bmatrix} 123 & 189 & 41 & 31 \\ * & 25 & 112 & 183 \\ 202 & 38 & 211 & 215 \\ 218 & 188 & 117 & 34 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} 48 & 16 & 194 & 55 \\ * & 237 & 85 & 225 \\ 108 & 218 & 83 & 186 \\ 154 & 71 & 135 & 139 \end{bmatrix}$$

我们选择的密钥如下：

$$key = 0xabf266f7a189b08e5db4a56ea585ae0f$$

2.5.2 结果记录

根据实验运行结果，记录如下表：(三组明文得到的结果都是一样的，对于 4 轮的情况，由于数据太长，只写了第一个)

实验组别	轮数	包含 MixColumns	异或结果 (XOR Sum)
1	3	是	$[0, 0, 0]$ (平衡)
2	4	是	$[213218948687531644049869954314427593478, \ldots]$ (不平衡)
3	4-31	否	$[0, 0, 0]$ (始终平衡)

Table 4: 不同配置下的 AES 异或实验结果

.1 附录：完整 S1 盒差分分布表 (DDT)

Table 5: DES S1 盒完整差分分布表

Input Xor \ Output Xor	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	64	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
01	0	0	0	6	0	2	4	4	0	10	12	4	10	6	2	4
02	0	0	0	0	8	0	4	4	4	0	6	8	6	12	6	4
03	14	4	2	2	10	6	4	2	6	4	4	0	2	2	2	0
04	0	0	0	6	0	10	10	6	0	4	6	4	2	8	6	2
05	4	8	6	2	2	4	4	2	0	4	4	0	12	2	4	6
06	0	4	2	4	8	2	6	2	8	4	4	2	4	2	0	12
07	2	4	10	4	0	4	8	4	2	4	8	2	2	2	4	4
08	0	0	0	12	0	8	8	4	0	6	2	8	8	2	2	4
09	10	2	4	0	2	4	6	0	2	2	8	0	10	0	2	12
0A	0	8	6	2	2	8	6	0	6	4	6	0	4	0	2	10
0B	2	4	0	10	2	2	4	0	2	6	2	6	6	4	2	12
0C	0	0	0	8	0	6	6	0	0	6	6	4	6	6	14	2
0D	6	6	4	8	4	8	2	6	0	6	4	6	0	2	0	2
0E	0	4	8	8	6	6	4	0	6	6	4	0	0	4	0	8
0F	2	0	2	4	4	6	4	2	4	8	2	2	2	6	8	8
10	0	0	0	0	0	0	2	14	0	6	6	12	4	6	8	6
11	6	8	2	4	6	4	8	6	4	0	6	6	0	4	0	0
12	0	8	4	2	6	6	4	6	6	4	2	6	6	0	4	0
13	2	4	4	6	2	0	4	6	2	0	6	8	4	6	4	6
14	0	8	8	0	10	0	4	2	8	2	2	4	4	8	4	0
15	0	4	6	4	2	2	4	10	6	2	0	10	0	4	6	4
16	0	8	10	8	0	2	2	6	10	2	0	2	0	6	2	6
17	4	4	6	0	10	6	0	2	4	4	4	6	6	6	2	0
18	0	6	6	0	8	4	2	2	2	4	6	8	6	6	2	2
19	2	6	2	4	0	8	4	6	10	4	0	4	2	8	4	0
1A	0	6	4	0	4	6	6	6	6	2	2	0	4	4	6	8
1B	4	4	2	4	10	6	6	4	6	2	2	4	2	2	4	2
1C	0	10	10	6	6	0	0	12	6	4	0	0	2	4	4	0
1D	4	2	4	0	8	0	0	2	10	0	2	6	6	6	14	0
1E	0	2	6	0	14	2	0	0	6	4	10	8	2	2	6	2
1F	2	4	10	6	2	2	2	8	6	8	0	0	0	4	6	4
20	0	0	0	10	0	12	8	2	0	6	4	4	4	2	0	12
21	0	4	2	4	4	8	10	0	4	4	10	0	4	0	2	8
22	10	4	6	2	2	8	2	2	2	2	6	0	4	0	4	10

Input Xor \ Output Xor	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
23	0	4	4	8	0	2	6	0	6	6	2	10	2	4	0	10
24	12	0	0	2	2	2	2	0	14	14	2	0	2	6	2	4
25	6	4	4	12	4	4	4	10	2	2	2	0	4	2	2	2
26	0	0	4	10	10	10	2	4	0	4	6	4	4	4	2	0
27	10	4	2	0	2	4	2	0	4	8	0	4	8	8	4	4
28	12	2	2	8	2	6	12	0	0	2	6	0	4	0	6	2
29	4	2	2	10	0	2	4	0	0	14	10	2	4	6	0	4
2A	4	2	4	6	0	2	8	2	2	14	2	6	2	6	2	2
2B	12	2	2	2	4	6	6	2	0	2	6	2	6	0	8	4
2C	4	2	2	4	0	2	10	4	2	2	4	8	8	4	2	6
2D	6	2	6	2	8	4	4	4	2	4	6	0	8	2	0	6
2E	6	6	2	2	0	2	4	6	4	0	6	2	12	2	6	4
2F	2	2	2	2	2	6	8	8	2	4	4	6	8	2	4	2
30	0	4	6	0	12	6	2	2	8	2	4	4	6	2	2	4
31	4	8	2	10	2	2	2	2	6	0	0	2	2	4	10	8
32	4	2	6	4	4	2	2	4	6	6	4	8	2	2	8	0
33	4	4	6	2	10	8	4	2	4	0	2	2	4	6	2	4
34	0	8	16	6	2	0	0	12	6	0	0	0	0	8	0	6
35	2	2	4	0	8	0	0	0	14	4	6	8	0	2	14	0
36	2	6	2	2	8	0	2	2	4	2	6	8	6	4	10	0
37	2	2	12	4	2	4	4	10	4	4	2	6	0	2	2	4
38	0	6	2	2	2	0	2	2	4	6	4	4	4	6	10	10
39	6	2	2	4	12	6	4	8	4	0	2	4	2	4	4	0
3A	6	4	6	4	6	8	0	6	2	2	6	2	2	6	4	0
3B	2	6	4	0	0	2	4	6	4	6	8	6	4	4	6	2
3C	0	10	4	0	12	0	4	2	6	0	4	12	4	4	2	0
3D	0	8	6	2	2	6	0	8	4	4	0	4	0	12	4	4
3E	4	8	2	2	2	4	4	14	4	2	0	2	0	8	4	4
3F	4	8	4	2	4	0	2	4	4	2	4	8	8	6	2	2

.2 附录

```
import pandas as pd
import numpy as np
#1、从DES算法的8个S盒中任选一个，计算其对应的差分分布表。
# 选择S1
S1 = [
    [14,  4, 13,  1,  2, 15, 11,  8,  3, 10,  6, 12,  5,  9,  0,  7],
    [0, 15,  7,  4, 14,  2, 13,  1, 10,  6, 12, 11,  9,  5,  3,  8],
    [4,  1, 14,  8, 13,  6,  2, 11, 15, 12,  9,  7,  3, 10,  5,  0],
    [15, 12,  8,  2,  4,  9,  1,  7,  5, 11,  3, 14, 10,  0,  6, 13]
]
def Sbox(x):
    row = (x>>5<<1) | (x&0b1)   # 第一位+最后一位决定行
    col = (x>>1) & 0b1111       # 中间四位决定列
```

```
    return S1[row][col]
#alpha遍历了6bit, beta遍历了4bit
DDT = [[0 for _ in range(16)] for _ in range(64)]
#遍历alpha
for alpha in range(64):
    #遍历所有x, 计算差分
    for x in range(64):
        beta = Sbox(x) ^ Sbox(x ^ alpha)  # 计算输出差分
        DDT[alpha][beta] += 1  # 更新DDT计数
print("差分分布表 (DDT) :")

df = pd.DataFrame(DDT)
df.index = [f"{i:02X}" for i in range(64)]
df.columns = [f"{i:X}" for i in range(16)]


df.index.name = "Input XOR"
df.columns.name = "Output XOR"

table = df.to_string(col_space=4, justify='center')

print(table)
#该DDT中概率最大的 (输入差分, 输出差分) 对应有多少种情况? 概率为多
    少?
DDT = np.array(DDT)
size = 64  # 输入空间大小


print("概率最大的前5个 (输入差分, 输出差分) 及其概率: ")
temp_ddt = DDT.copy()

counts = sorted(list(set(temp_ddt.flatten())), reverse=True)
top_5_counts = counts[:5]

print(f"{'排名':<4} | {'计数':<4} | {'概率':<10} | {'(输入差分, 输出
    差分)列表'}")
print("-" * 60)

for rank, count in enumerate(top_5_counts, 1):
    prob = count / 64.0

    coords = np.argwhere(temp_ddt == count)

    res = ", ".join([f"(0x{r:02X}, 0x{c:X})" for r, c in coords])

    print(f"{rank:>3}  | {count:>4} | {prob:<10.6f} | {res}")

ddt = np.array(DDT).flatten()
```

```
# 该 S 盒共有多少种概率传播？每种概率传播的概率是多少？频数不为0，都
    是可能的传播
eff_ddt = ddt[ddt > 0]

eff_ddt = sorted(list(set(eff_ddt)))

print(f"该 S 盒共有 {len(eff_ddt)} 种概率传播：")
print("-" * 30)
for count in eff_ddt:
    prob = count / 64.0
    print(f"计数 {count:2} -> 概率 {prob:.4f} (即 {count}/64)")
#对该S盒，输入差分非0，输出差分可能为0吗？为什么？
# 获取除第一行外的第一列数据
data = df.iloc[1:, 0]

# 使用布尔索引筛选出计数大于 0 的项
zero = data[data > 0]
print("输入差分非 0 但输出差分为 0 的情况：")
for alpha, count in zero.items():
    print(f"输入差分{alpha}: 有 {count} 个输入导致输出差分为 0")
```

```
#AES实现来自cryptohack
class AES:
    def __init__(self, N_ROUNDS=10):
        self.N_ROUNDS = N_ROUNDS

    def bytes2matrix(self, text):
        """ Converts a 16-byte array into a 4x4 matrix.  """
        return [list(text[i:i+4]) for i in range(0, len(text), 4)]

    def matrix2bytes(self, matrix):
        """ Converts a 4x4 matrix into a 16-byte array.  """
        return bytes([byte for row in matrix for byte in row])

    def add_round_key(self, s, k):
        return [[ss ^ kk for ss, kk in zip(s_row, k_row)] for s_row,
            k_row in zip(s, k)]


    s_box = (
        0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0
            x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
        0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0
            xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
        0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0
            xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
        0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0
            x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
        0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0
```

```
        xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0
        xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0
        x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0
        xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0
        x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0
        xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0
        xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0
        xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0
        x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0
        x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0
        x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0
        x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16,
)

inv_s_box = (
    0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF, 0x40, 0
        xA3, 0x9E, 0x81, 0xF3, 0xD7, 0xFB,
    0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E, 0
        x43, 0x44, 0xC4, 0xDE, 0xE9, 0xCB,
    0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C, 0
        x95, 0x0B, 0x42, 0xFA, 0xC3, 0x4E,
    0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B, 0
        xA2, 0x49, 0x6D, 0x8B, 0xD1, 0x25,
    0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0
        x5C, 0xCC, 0x5D, 0x65, 0xB6, 0x92,
    0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E, 0x15, 0
        x46, 0x57, 0xA7, 0x8D, 0x9D, 0x84,
    0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A, 0xF7, 0xE4, 0
        x58, 0x05, 0xB8, 0xB3, 0x45, 0x06,
    0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF, 0
        xBD, 0x03, 0x01, 0x13, 0x8A, 0x6B,
    0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA, 0x97, 0xF2, 0
        xCF, 0xCE, 0xF0, 0xB4, 0xE6, 0x73,
    0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD, 0x35, 0x85, 0xE2, 0xF9, 0
        x37, 0xE8, 0x1C, 0x75, 0xDF, 0x6E,
    0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F, 0xB7, 0
        x62, 0x0E, 0xAA, 0x18, 0xBE, 0x1B,
    0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB, 0
```

```
        xC0, 0xFE, 0x78, 0xCD, 0x5A, 0xF4,
    0x1F, 0xDD, 0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0
        x10, 0x59, 0x27, 0x80, 0xEC, 0x5F,
    0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D, 0xE5, 0
        x7A, 0x9F, 0x93, 0xC9, 0x9C, 0xEF,
    0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB, 0
        xBB, 0x3C, 0x83, 0x53, 0x99, 0x61,
    0x17, 0x2B, 0x04, 0x7E, 0xBA, 0x77, 0xD6, 0x26, 0xE1, 0x69, 0
        x14, 0x63, 0x55, 0x21, 0x0C, 0x7D,
)
def sub_bytes(self, s, sbox=None):
    if sbox is None:
        sbox = self.s_box
    return [[sbox[byte] for byte in row] for row in s]
def shift_rows(self, s):
    s[0][1], s[1][1], s[2][1], s[3][1] = s[1][1], s[2][1], s
        [3][1], s[0][1]
    s[0][2], s[1][2], s[2][2], s[3][2] = s[2][2], s[3][2], s
        [0][2], s[1][2]
    s[0][3], s[1][3], s[2][3], s[3][3] = s[3][3], s[0][3], s
        [1][3], s[2][3]


def inv_shift_rows(self, s):
    s[0][1], s[1][1], s[2][1], s[3][1] = s[3][1], s[0][1], s
        [1][1], s[2][1]
    s[0][2], s[1][2], s[2][2], s[3][2] = s[2][2], s[3][2], s
        [0][2], s[1][2]
    s[0][3], s[1][3], s[2][3], s[3][3] = s[1][3], s[2][3], s
        [3][3], s[0][3]


# learned from http://cs.ucsb.edu/~koc/cs178/projects/JT/aes.c
def xtime(self, a):
    return (((a << 1) ^ 0x1B) & 0xFF) if (a & 0x80) else (a << 1)


def mix_single_column(self, a):
    # see Sec 4.1.2 in The Design of Rijndael
    t = a[0] ^ a[1] ^ a[2] ^ a[3]
    u = a[0]
    a[0] ^= t ^ self.xtime(a[0] ^ a[1])
    a[1] ^= t ^ self.xtime(a[1] ^ a[2])
    a[2] ^= t ^ self.xtime(a[2] ^ a[3])
    a[3] ^= t ^ self.xtime(a[3] ^ u)


def mix_columns(self, s):
    for i in range(4):
```

```
            self.mix_single_column(s[i])


    def inv_mix_columns(self,s):
        # see Sec 4.1.3 in The Design of Rijndael
        for i in range(4):
            u = self.xtime(self.xtime(s[i][0] ^ s[i][2]))
            v = self.xtime(self.xtime(s[i][1] ^ s[i][3]))
            s[i][0] ^= u
            s[i][1] ^= v
            s[i][2] ^= u
            s[i][3] ^= v

        self.mix_columns(s)




    def expand_key(self,master_key):
        """
        Expands and returns a list of key matrices for the given
           master_key.
        """

        # Round constants https://en.wikipedia.org/wiki/
           AES_key_schedule#Round_constants
        r_con = (
            0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40,
            0x80, 0x1B, 0x36, 0x6C, 0xD8, 0xAB, 0x4D, 0x9A,
            0x2F, 0x5E, 0xBC, 0x63, 0xC6, 0x97, 0x35, 0x6A,
            0xD4, 0xB3, 0x7D, 0xFA, 0xEF, 0xC5, 0x91, 0x39,
        )

        # Initialize round keys with raw key material.
        key_columns = self.bytes2matrix(master_key)
        iteration_size = len(master_key) // 4

        # Each iteration has exactly as many columns as the key
           material.
        i = 1
        while len(key_columns) < (self.N_ROUNDS + 1) * 4:
            # Copy previous word.
            word = list(key_columns[-1])

            # Perform schedule_core once every "row".
            if len(key_columns) % iteration_size == 0:
                # Circular shift.
                word.append(word.pop(0))
                # Map to S-BOX.
```

```
                word = [self.s_box[b] for b in word]
                # XOR with first byte of R-CON, since the others
                    bytes of R-CON are 0.
                word[0] ^= r_con[i]
                i += 1
            elif len(master_key) == 32 and len(key_columns) %
                iteration_size == 4:
                # Run word through S-box in the fourth iteration when
                    using a
                # 256-bit key.
                word = [self.s_box[b] for b in word]

            # XOR with equivalent word from previous iteration.
            word = bytes(i^j for i, j in zip(word, key_columns[-
                iteration_size]))
            key_columns.append(word)

        # Group key words in 4x4 byte matrices.
        return [key_columns[4*i : 4*(i+1)] for i in range(len(
            key_columns) // 4)]

    def encrypt(self, key, plaintext):
        round_keys = self.expand_key(key)

        # Convert plaintext to state matrix
        state = self.bytes2matrix(plaintext)

        # Initial add round key step
        state = self.add_round_key(state, round_keys[0])

        for i in range(1, self.N_ROUNDS+1):
            state = self.sub_bytes(state)
            self.shift_rows(state)
            self.mix_columns(state)
            state = self.add_round_key(state, round_keys[i])

        # Convert state matrix to ciphertext
        return self.matrix2bytes(state)
    def encrypt1(self, key, plaintext):
        round_keys = self.expand_key(key)

        # Convert plaintext to state matrix
        state = self.bytes2matrix(plaintext)

        # Initial add round key step
        state = self.add_round_key(state, round_keys[0])

        for i in range(1, self.N_ROUNDS+1):
            state = self.sub_bytes(state)
```

```
            self.shift_rows(state)
            state = self.add_round_key(state, round_keys[i])

        # Convert state matrix to ciphertext
        return self.matrix2bytes(state)
```

References

[1] David Wong. Block breakers.

[2] 知乎. Aes 积分攻击.